

Lab 4 — Versioning, Feature Store & Lineage

Track A (tabular fraud-detection) · Week 4 · DS5619 Machine Learning Systems Operations

1. Student Identification & Run Metadata

- **Student Roll Number / ID:** 142301024
 - **Deterministic Salt:** week04
 - **Computed Random Seed:** 1413445892 (via `seed_from_student_id("142301024", salt="week04")`)
 - **Dataset Revisions:**
 - `data/v1/transactions.csv` : 500 rows (Base schema)
 - `data/v2/transactions.csv` : 125 rows (Breaking schema change)
-

2. Lab Overview & Architecture

In production Machine Learning systems, data pipelines are subject to frequent upstream schema changes, evolving business logic, and continuous data streams. This lab implements a lightweight, dependency-free **Feature Store and Lineage Registry** (`mini_feature_store.py`) that models core MLOps data management concepts:

1. **Content-Hash Raw Data Versioning:** Deterministic, idempotent raw data versioning based on SHA-256 content hashes (similar to DVC/Git LFS).
2. **Feature Store & Feature Groups:** Aggregated, entity-level feature tables (`card_activity`) computed from raw feeds with standardized schemas.
3. **Upstream Schema Evolution & Invariance:** Seamlessly absorbing breaking upstream schema changes (e.g., currency units, renamed columns) through normalization while guaranteeing immutable feature store contracts.
4. **End-to-End Lineage Tracking:** Explicit metadata provenance linking every feature group revision directly back to the exact raw data version and transformation logic that produced it.



3. Quickstart & Execution

3.1 Setup Environment

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

3.2 Generate Personalized Dataset

```
python generate_for_student.py --student-id 142301024
```

This overwrites `data/v1/transactions.csv` (500 records) and `data/v2/transactions.csv` (125 records) deterministically seeded from roll number `142301024`.

3.3 Execute the Feature Store Pipeline

```
python src/run_pipeline.py
```

This snapshots raw datasets into `.feature_store/raw_versions/`, builds normalized feature groups into `.feature_store/feature_groups/`, asserts snapshot idempotency, and compiles `lineage_report.json`.

3.4 Run Smoke Tests & Verification

```
pytest tests/ -v
```

4. Algorithms & Implementation Logic

The core logic is implemented in `src/mini_feature_store.py` across four primary functions:

4.1 Part 1 — Raw Data Versioning (`snapshot_raw_version`)

```
def snapshot_raw_version(input_path: str, registry_dir: str) -> str:
```

Objective

To register a raw dataset file as an immutable version under `registry_dir/raw_versions/{version_id}/` in an **idempotent** manner.

Algorithm & Step-by-Step Logic

1. **Cryptographic Content Hashing:**
2. Computes the SHA-256 hash of the input file in 8192-byte chunks via `content_hash(input_path)`.
3. Streaming in chunks ensures memory efficiency even for large datasets.
4. **Idempotency Scan:**
5. Inspects existing manifests under `registry_dir/raw_versions/*/manifest.json`.
6. If an existing manifest has `manifest["content_hash"] == file_hash`, the function immediately returns that `version_id` without creating duplicate directories or mutating state.
7. **Monotonic Version Allocation:**
8. If the hash is new, invokes `_next_version_id(...)` which scans existing directory names matching `v(\d+)` and increments the maximum index (e.g., `v1` $\rightarrow$ `v2`).
9. **Manifest Assembly & Persistence:**
10. Reads the raw CSV file to inspect headers and row count.
11. Generates a structured `manifest.json` containing:
 - `version_id` : Allocated identifier (e.g. `"v1"`)
 - `source_path` : Original file path
 - `content_hash` : Computed SHA-256 hex digest
 - `columns` : Extracted header list
 - `row_count` : Number of data rows
 - `created_at` : UTC ISO8601 timestamp (`_now()`)
12. Writes `manifest.json` to `registry_dir/raw_versions/{version_id}/manifest.json`.
13. **Return:** Returns the allocated `version_id` string.

4.2 Part 2 — Feature Engineering & Schema Normalization (`build_features`)

```
def build_features(rows: list[dict]) -> list[dict]:
```

Objective

Given a list of transaction row dictionaries (originating from either v1 or v2 schema), compute aggregate entity-level features per unique `card_id`.

Upstream Schema Evolution Context

Field Attribute	Schema v1 (data/v1/)	Schema v2 (data/v2/)	Normalization Action
Transaction Amount	amount (float string, base units e.g. "67.45")	amount_minor_units (integer cents e.g. "6745")	Convert v2 cents to base units: $\text{\text{amount}} = \frac{\text{\text{amount_minor_units}}}{100.0}$
Country Information	country (e.g. "US")	country_code (e.g. "US")	Handled without affecting core card aggregates
Device Information	Not present	device_fingerprint (e.g. "dev_000123")	Added upstream; filtered out from card aggregate
Card Presence	"True" / "False" string	"True" / "False" string	Evaluated as boolean truthy if == "True"
Event Timestamp	ISO8601 UTC string	ISO8601 UTC string	Track max timestamp per card_id

Algorithm & Mathematical Definitions

- Grouping:** Group all raw transaction dictionaries by entity key `card_id` .
- Schema Detection & Normalization:**
- For each transaction row i in $\{1, \dots, N\}$ for a given card:
$$\text{\text{amount}}_i = \begin{cases} \frac{\text{\text{float}}(\text{\text{row}}_i[\text{"amount_minor_units"}])}{100.0} & \text{if "amount_minor_units" in } \text{\text{row}}_i \\ \text{\text{float}}(\text{\text{row}}_i[\text{"amount"}]) & \text{otherwise} \end{cases}$$
- Aggregate Computations:**
- Transaction Count** (N):
$$N = \text{\text{txn_count}} = |\text{\text{txns}}|$$
- Average Transaction Amount** (μ):
$$\mu = \text{\text{avg_amount}} = \text{\text{round}}\left(\frac{1}{N} \sum_{i=1}^N \text{\text{amount}}_i, 2\right)$$
- Maximum Transaction Amount** (M):
$$M = \text{\text{max_amount}} = \text{\text{round}}\left(\max_{1 \leq i \leq N} \text{\text{amount}}_i, 2\right)$$
- Percentage Card Present** (P):
$$P = \text{\text{pct_card_present}} = \text{\text{round}}\left(\frac{1}{N} \sum_{i=1}^N \mathbb{1}(\text{\text{card_present}}_i = \text{"True"}), 3\right)$$
- Event Time** ($T_{\max}$):
$$T_{\max} = \text{\text{event_time}} = \max_{1 \leq i \leq N} (\text{\text{timestamp}}_i)$$
 (Because timestamps are ISO8601 formatted, lexicographical string comparison is monotonic and correct).
- Return:** Returns a list of feature dictionaries (one per distinct `card_id`).

4.3 Part 3 — Feature Group Registration (register_feature_group)

```
def register_feature_group(
    name: str,
    feature_rows: list[dict],
    source_version_id: str,
    registry_dir: str,
    transform_version: str = "v1"
) -> str:
```

Objective

To register a new, immutable version of the feature group under `registry_dir/feature_groups/{name}/{fg_version_id}/`.

Algorithm & Step-by-Step Logic

1. **Version Isolation:**
2. Scans `registry_dir/feature_groups/{name}/` using `_next_version_id` to allocate the next incremental version id (e.g. `v1`, `v2`).
3. Ensures previous versions are **never overwritten or mutated**, preserving historical feature data for model reproducibility.
4. **Feature Data Storage:**
5. Creates the destination directory `registry_dir/feature_groups/{name}/{fg_version_id}/`.
6. Serializes and writes `feature_rows` into `features.json`.
7. **Lineage Manifest Registration:**
8. Extracts the feature schema as the sorted list of keys in `feature_rows[0]`.
9. Writes `manifest.json` with metadata:
 - `feature_group_version_id` : Newly assigned version (e.g. `"v1"`)
 - `name` : Feature group name (e.g. `"card_activity"`)
 - `source_raw_version_id` : Pointer to source raw version (e.g. `"v1"`)
 - `transform_version` : Transformation code version (e.g. `"v1"`)
 - `schema` : `["avg_amount", "card_id", "event_time", "max_amount", "pct_card_present", "txn_count"]`
 - `row_count` : Number of feature rows (unique cards)
 - `created_at` : UTC ISO8601 timestamp
10. **Return:** Returns `fg_version_id`.

4.4 Part 4 — Lineage Tracing (`get_lineage`)

```
def get_lineage(name: str, fg_version_id: str, registry_dir: str) -> dict:
```

Objective

To trace the provenance graph of a given feature group version back to its raw data source and return the combined chain.

Algorithm & Step-by-Step Logic

1. **Read Feature Group Manifest:**
2. Loads `registry_dir/feature_groups/{name}/{fg_version_id}/manifest.json`.
3. **Resolve Raw Source Pointer:**
4. Extracts `source_raw_version_id` from the feature group manifest.
5. Loads `registry_dir/raw_versions/{source_raw_version_id}/manifest.json`.
6. **Assemble Lineage Chain:**
7. Combines both manifest dictionaries into a structured lineage payload: `json { "feature_group": { ...feature group manifest... }, "raw_source": { ...raw data manifest... } }`

8. **Error Handling:** Missing files naturally raise `FileNotFoundError` .

5. End-to-End Pipeline Workflow (`src/run_pipeline.py`)

The pipeline driver executes the complete end-to-end lifecycle:

1. **Step 1 (Raw v1 Ingestion):** Snapshots `data/v1/transactions.csv` $\rightarrow$ `raw_versions/v1` .
2. **Step 2 (Feature Group v1 Creation):** Transforms v1 rows $\rightarrow$ registers `card_activity/v1` (383 card rows).
3. **Step 3 (Raw v2 Ingestion):** Snapshots `data/v2/transactions.csv` $\rightarrow$ `raw_versions/v2` .
4. **Step 4 (Feature Group v2 Creation):** Normalizes and transforms v2 rows $\rightarrow$ registers `card_activity/v2` (115 card rows).
5. **Step 5 (Idempotency Verification):** Re-runs `snapshot_raw_version(v1_path)` and asserts that it returns `v1` without creating extra versions.
6. **Step 6 (Lineage Report Generation):** Resolves lineage for `card_activity` v1 and v2, and saves the full report to `lineage_report.json` .

6. Registry Structure & Artifacts

6.1 Directory Tree

```
week-4/
├── .feature_store/
│   ├── feature_groups/
│   │   └── card_activity/
│   │       ├── v1/
│   │       │   ├── features.json      # 383 aggregated card feature records
│   │       │   └── manifest.json      # Schema, row_count=383, source_raw_version_id="v1"
│   │       └── v2/
│   │           ├── features.json      # 115 aggregated card feature records
│   │           └── manifest.json      # Schema, row_count=115, source_raw_version_id="v2"
│   └── raw_versions/
│       ├── v1/
│       │   └── manifest.json          # 500 rows, hash=3ae42e3b..., columns=[...amount...]
│       └── v2/
│           └── manifest.json          # 125 rows, hash=2e5411f8..., columns=[...amount_minor_units...]
├── data/
│   ├── v1/
│   │   └── transactions.csv          # 500 raw transactions (v1 schema)
│   └── v2/
│       └── transactions.csv          # 125 raw transactions (v2 schema)
├── lineage_report.json               # Complete lineage report generated by pipeline
├── NOTES.md                         # Student metadata and manifest comparison notes
├── requirements.txt                  # Project dependencies
├── src/
│   ├── mini_feature_store.py         # Core feature store implementation
│   └── run_pipeline.py               # Pipeline driver
└── tests/
    └── test_smoke.py                 # Pytest verification suite
```

7. Results & Analysis for Student ID 142301024

7.1 Manifest Comparison Summary

Metric / Attribute	Raw Version v1	Raw Version v2	Feature Group v1	Feature Group v2
Version ID	v1	v2	v1	v2
Row Count	500 transactions	125 transactions	383 unique cards	115 unique cards
Content Hash (SHA-256)	3ae42e3b3de...	2e5411f8266...	N/A	N/A
Source Raw Pointer	N/A	N/A	v1	v2
Transform Version	N/A	N/A	v1	v1
Schema	8 columns (amount , country)	9 columns (amount_minor_units , country_code , device_fingerprint)	6 features (avg_amount , card_id , event_time , max_amount , pct_card_present , txn_count)	6 features (avg_amount , card_id , event_time , max_amount , pct_card_present , txn_count)

7.2 Why Unit Normalization is Critical

Without schema detection and the `amount_minor_units / 100.0` transformation: - A transaction of \(\$50.00 in v1 is stored as `50.0` . - The identical transaction in v2 is stored as `5000` (cents). - If aggregated directly without conversion, the computed `avg_amount` in v2 would be **100x larger** than in v1. - Machine Learning models trained on v1 features would experience extreme distribution shift and prediction failure when evaluated against v2 features. - Normalization ensures **semantic equivalence and scale invariance** across feature store revisions.

7.3 Complete `lineage_report.json` Output

```

{
  "feature_group": "card_activity",
  "v1": {
    "feature_group": {
      "feature_group_version_id": "v1",
      "name": "card_activity",
      "source_raw_version_id": "v1",
      "transform_version": "v1",
      "schema": [
        "avg_amount",
        "card_id",
        "event_time",
        "max_amount",
        "pct_card_present",
        "txn_count"
      ],
      "row_count": 383,
      "created_at": "2026-08-26T15:00:57.565265+00:00"
    },
    "raw_source": {
      "version_id": "v1",
      "source_path": ".../week-4/data/v1/transactions.csv",
      "content_hash": "3ae42e3b3de77ad2238bebc123cd1af2180872e65ee2b43813f84914bd251022",
      "columns": [
        "transaction_id",
        "card_id",
        "timestamp",
        "amount",
        "merchant_category",
        "country",
        "card_present",
        "is_fraud"
      ],
      "row_count": 500,
      "created_at": "2026-08-26T15:00:57.560379+00:00"
    }
  },
  "v2": {
    "feature_group": {
      "feature_group_version_id": "v2",
      "name": "card_activity",
      "source_raw_version_id": "v2",
      "transform_version": "v1",
      "schema": [
        "avg_amount",
        "card_id",
        "event_time",
        "max_amount",
        "pct_card_present",
        "txn_count"
      ],
      "row_count": 115,
      "created_at": "2026-08-26T15:00:57.567558+00:00"
    },
    "raw_source": {
      "version_id": "v2",
      "source_path": ".../week-4/data/v2/transactions.csv",
      "content_hash": "2e5411f82667ed59adb61d14cf200b36009a739156d1323b397882da874fc140",
      "columns": [
        "transaction_id",

```

```

        "card_id",
        "timestamp",
        "merchant_category",
        "card_present",
        "is_fraud",
        "device_fingerprint",
        "country_code",
        "amount_minor_units"
    ],
    "row_count": 125,
    "created_at": "2026-08-26T15:00:57.565917+00:00"
}
}
}

```

8. Verification & Smoke Test Results

All 8 smoke test suites pass cleanly:

```

$ pytest tests/ -v
===== test session starts =====
platform darwin -- Python 3.12.7, pytest-8.3.3, pluggy-1.6.0
rootdir: /Users/vivekchaithayamekarthi/Desktop/ds5619-mlops/week-4
collected 8 items

tests/test_smoke.py::test_snapshot_raw_version_creates_manifest PASSED [ 12%]
tests/test_smoke.py::test_snapshot_raw_version_is_idempotent PASSED [ 25%]
tests/test_smoke.py::test_snapshot_raw_version_detects_change PASSED [ 37%]
tests/test_smoke.py::test_build_features_v1_schema PASSED [ 50%]
tests/test_smoke.py::test_build_features_v2_schema PASSED [ 62%]
tests/test_smoke.py::test_register_feature_group_does_not_overwrite PASSED [ 75%]
tests/test_smoke.py::test_get_lineage_traces_to_raw_source PASSED [ 87%]
tests/test_smoke.py::test_full_pipeline_runs_and_writes_lineage_report PASSED [100%]

===== 8 passed in 0.08s =====

```

9. Submission Checklist

- [x] `data/` generated with `python generate_for_student.py --student-id 142301024`.
- [x] `snapshot_raw_version` verified genuinely idempotent (re-running does not duplicate versions).
- [x] `build_features` correctly normalizes `amount_minor_units` and calculates exact metrics.
- [x] `register_feature_group` immutably registers `v1` and `v2` without overwriting history.
- [x] `get_lineage` resolves full provenance back to raw version manifests.
- [x] `NOTES.md` documents student ID, seed, manifest comparisons, and normalization rationale.
- [x] `README.md` and `README.pdf` updated with comprehensive technical and architectural documentation.

```

git add -A
git commit -m "Week 4: mini feature store + lineage"
git tag week04-submit
git push origin main --tags

```